A connected chordal counterexample to the clique-tree genus formula

Disproof of Conjecture 00000003484

Galaxy-0
AI-assisted submission

14 October 2026

Abstract

The conjecture defines the nodes of a clique tree to be its maximal-clique bags, but subtracts a proposed genus from their number and characterizes zero genus by interval representability. The connected six-vertex 3-sun has four maximal cliques and a clique tree with four bags, yet is not an interval graph. Consequently no genus value can satisfy the two asserted clauses simultaneously. We give a self-contained proof and a Lean 4.31.0 verification, including the graph hypotheses, both counts, the actual interval-intersection semantics, and the contradiction.

1 The assertion being refuted

The source defines a clique tree as a tree of the maximal cliques of a chordal graph, with those bags as nodes. Its proposed conditions include

$$B = M - h, \qquad h = 0 \iff G \text{ is an interval graph}, \tag{1}$$

where B counts bags and M counts maximal cliques. It also asserts an explicit recursion involving clique-tree embedding counts. Refuting the displayed necessary conditions refutes the conjunction, without interpreting that recursion.

The source does not define "Helly covering genus". We do not assign a new meaning to it or assume a value for it. Instead we prove that *no* value can make both conditions hold on the example. We use the conventional clique tree, with each maximal clique appearing once, exactly as in the stated definition. A compressed decomposition with different nodes would change the problem. The Chinese wording may conjoin the recursion with interval representability on the right of the equivalence; that reading still implies that $h = 0$ forces interval representability, which is all the contradiction uses.

2 The graph and its clique tree

Let the vertices be a, b, c, x, y, z. The vertices a, b, c induce a triangle; x, y, z are pairwise nonadjacent, and their respective neighborhoods are

$$N(x) = \{a, b\}, \qquad N(y) = \{b, c\}, \qquad N(z) = \{a, c\}.$$

There are no other edges. Thus this is a finite simple connected graph. Every pair of vertices is joined by a path of length at most two.

Lemma 1. *This graph is chordal, and its maximal cliques are exactly*

$$C = \{a, b, c\}, \quad X = \{a, b, x\}, \quad Y = \{b, c, y\}, \quad Z = \{a, c, z\}.$$

Proof. Consider an induced simple cycle of length at least four. If it contains an outer vertex, the two distinct cycle neighbors of that vertex are its two central neighbors. Those neighbors are adjacent, and are not consecutive in the cycle, so this is a chord. A cycle avoiding all outer vertices would require at least four distinct vertices in the three-vertex center, which is impossible. This proves chordality directly, without appealing to a perfect-elimination-order equivalence.

A clique containing x, y, or z is contained in X, Y, or Z, respectively. A clique containing no outer vertex is contained in C. The four displayed sets are cliques. For each one, every vertex outside it misses some vertex inside it, so it is maximal. Hence the list is complete and has no repetitions. □

Make a star with center C and leaves X, Y, Z. It is connected and has no simple cycle: each leaf has just one neighbor. The bags containing a are C, X, Z; those containing b are C, X, Y; those containing c are C, Y, Z; each outer vertex occurs in a single bag. In each case these bags induce a connected subtree. This is the running-intersection property, so the star is a clique tree. Both independently certified counts are therefore

$$B = 4, \qquad M = 4. \tag{2}$$

3 The interval obstruction

An interval graph is the intersection graph of nonempty closed real intervals. This usual convention suffices; allowing arbitrary interval endpoint conventions does not enlarge the finite class. Indeed, choose a point in each nonempty represented interval and one point in every intersection corresponding to an edge. For each vertex take the closed convex hull of its finitely many chosen points. It remains inside the original interval, preserves every edge, and creates no nonedge. Thus a finite representation by arbitrary intervals yields one by closed bounded intervals.

Lemma 2. *The 3-sun has no interval representation.*

Proof. Suppose its six vertices have representing intervals. The three intervals for x, y, z are pairwise disjoint, so they have a left-to-right order. If x is the middle outer interval, the central interval for c meets the two extreme outer intervals y, z but must miss x. This is impossible: an interval meeting intervals on both sides of a nonempty middle interval contains points across it and therefore meets it. If y is in the middle, use a instead; if z is in the middle, use b. These three cases exhaust the possible middle vertices. □

Theorem 1. *Conjecture 00000003484 is false.*

Proof. The graph is chordal and has the exhibited clique tree. Substituting (2) into (1) gives $4 = 4-h$, hence $h = 0$. The zero-genus implication would make it an interval graph, contradicting the preceding lemma. The argument uses ordinary numerical subtraction. It also holds for truncated natural-number subtraction: $4 = 4 \mathbin{\dot{-}} h$ implies $h = 0$ because $4 > 0$. □

4 Machine-checked semantics and scope

The project is self-contained and imports `Std`; no Mathlib dependency is needed. Vertices are `Fin 6`, with adjacency defined by the actual nine undirected edges. The core predicates and their bridges are explicit:

- `InducedCycle` uses an injective cyclic vertex map and adjacency if and only if cyclic consecutiveness. `threeSun_chordal` excludes such cycles of *every* length at least four, rather than testing bounded lengths or defining chordality as an unproved surrogate.

- `MaximalClique` quantifies over all vertex subsets. `maximal_cliques_complete` proves the four displayed bags enumerate precisely those sets. Their list is complete and without duplicates.
- `CliqueTree` requires symmetric irreflexive edges, connectedness, absence of simple cycles of every length, complete and injective maximal-clique bags, and paths within the bags containing each graph vertex. The explicit star satisfies all fields. Its node list is independently complete and without duplicates.
- An endpoint type carries an arbitrary total transitive relation. `Intersects` means there exists an actual point in both closed intervals. `intersects_iff_endpoints` proves the endpoint-overlap criterion from nonemptiness. `not_interval` then excludes representations on *any* such line, in particular the real line. No real-analysis theorem is assumed: the only properties needed are totality and transitivity, which real order has.
- `full_counterexample` combines connectedness, chordality, the genuine clique tree, both counts, non-representability, and the incompatible genus clauses for both natural and integer values. The formal genus statements cover these two types. The same elementary cancellation for other usual number systems is a written argument, not a separate Lean theorem.

The closed-interval normalization paragraph is a written finite argument; Lean directly covers the conventional nonempty closed-interval definition. The formalization does not define or evaluate the unspecified embedding recursion or propose a replacement genus. Neither is needed to refute the already inconsistent necessary conditions.

5 Verification and reproducibility

The pinned compiler is Lean 4.31.0. The package includes source, Lake configuration, an axiom audit, and reproducible commands. A separate source-only build with warnings treated as errors and a separate `leanchecker` kernel replay pass. Authored theorem audits use only the standard Lean axioms `propext`, `Classical.choice`, and `Quot.sound`, where needed. There are no proof holes, unchecked decision tactics, or user-declared axioms.

An independent Python program enumerates all vertex subsets and the 1,800 ordered possible cycles of lengths four through six. It confirms the maximal cliques, lack of induced long cycles, and running intersection. These are finite cross-checks; they are not substitutes for the universal Lean proofs. The included independent review is an internal mathematical/build audit, not organizer acceptance.

Source and eligibility

Source: Conjecture 00000003484, bilingual statement. The verified source Git blob is: `6e8283ae36deff4684090880c4d608259201b28e`. Evidence records main commit: `cce2d90cb298de9595798eb674d2c80712100532` and checks on 14 October 2026: metadata has neither proven nor disproven set, no solution directory exists, and all-state full-ID and keyword PR searches show no competing submission for this conjecture. These observations do not establish organizer acceptance or guarantee first-solver credit. Publication must recheck eligibility.